

CS376 – HW2 Report

학번 비공개 Sim Hyeonseong

[Task]

To improve training accuracy of the model, I applied polynomial basis function of degree 2 on X . The shape of the original X is (100, 3) where 100 means the number of samples, and 3 means the number of features: $[1 \ x_1 \ x_2]$ (1 represents the vector using for bias term). By defining the function, `polynomial(X)`, I introduced new variable X_{poly} and changed the shape of X into (100, 6). The matrix X_{poly} has 6 features: $[1 \ x_1 \ x_2 \ x_1^2 \ x_1x_2 \ x_2^2]$, and the matrix θ_{poly} which is the matrix for weights has 6 elements in it. Then, I trained logistic regression with X_{poly} and θ_{poly} with other hyperparameters same with the original training.

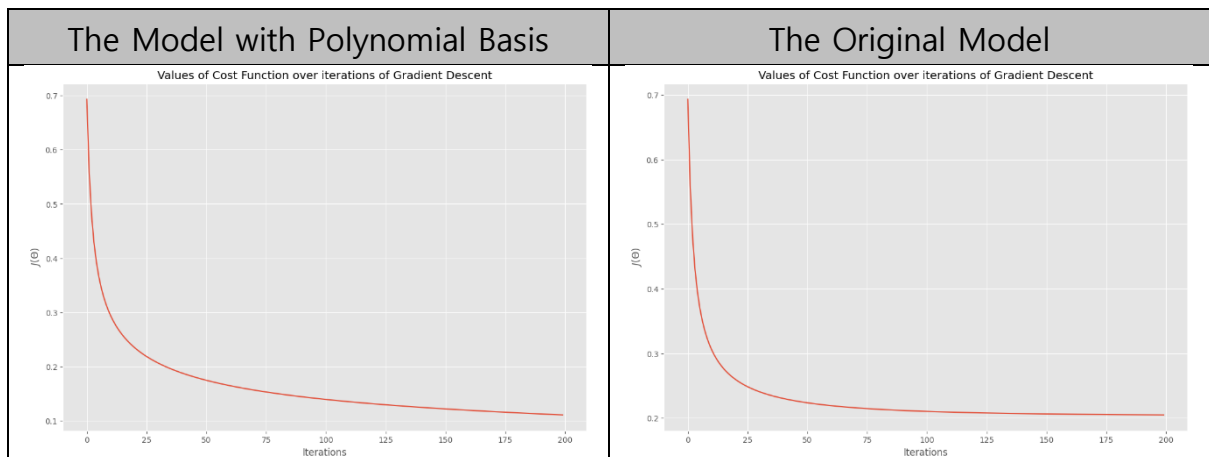
```
1 def polynomial(X):
2     x1 = X[:, 1]
3     x2 = X[:, 2]
4     X_poly = np.column_stack((X, x1**2, x1*x2, x2**2))
5     return X_poly
```

```
1 X_poly = polynomial(X)
2 theta_init_poly = np.zeros((X_poly.shape[1], 1))
3 theta_poly, costs_poly = gradient_descent(X_poly, y, theta_init_poly, 1, 200)
```

I plotted the graph of cost vs iteration for polynomial basis.

```
1 plt.plot(costs_poly)
2 plt.xlabel("Iterations")
3 plt.ylabel("$J(W\theta)$")
4 plt.title("Values of Cost Function over iterations of Gradient Descent");
```

The following two figures are cost vs iteration graphs of the model with polynomial basis (left figure) and the original model (right figure).



According to the graphs above, the model with polynomial basis shows lower cost (almost 0.1) at iteration 200 than the original one (almost 0.2).

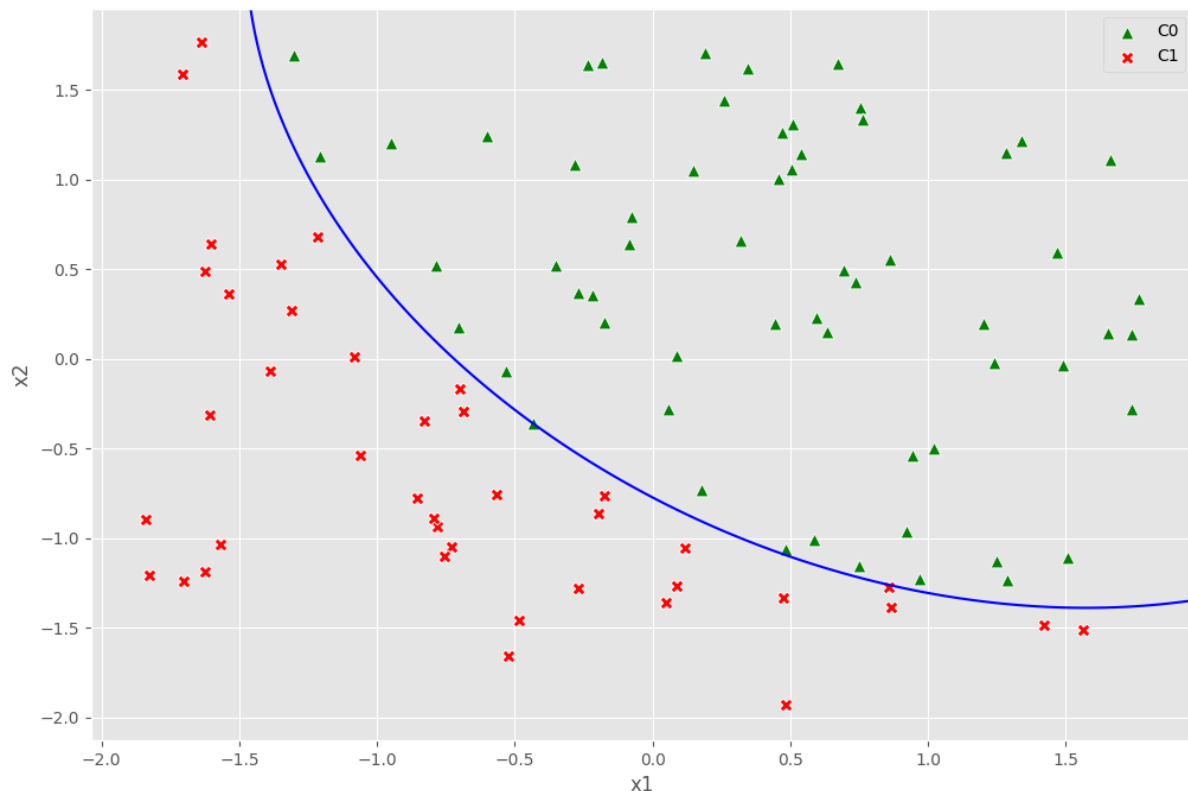
```

1 sns.scatterplot(x = X[class_c0[:, 0], 1],
2                 y = X[class_c0[:, 0], 2],
3                 marker = "^",
4                 color = "green",
5                 s = 60)
6 ax = sns.scatterplot(x = X[class_c1[:, 0], 1],
7                     y = X[class_c1[:, 0], 2],
8                     marker = "X",
9                     color = "red",
10                    s = 60)
11
12 ax.legend(["C0", "C1"])
13 ax.set(xlabel="x1", ylabel="x2")
14
15 resolution = 0.02
16 xs = np.arange(np.min(X[:, 1])-0.2, np.max(X[:, 1])+0.2, resolution)
17 ys = np.arange(np.min(X[:, 2])-0.2, np.max(X[:, 2])+0.2, resolution)
18 xx, yy = np.meshgrid(xs, ys)
19
20 X_grid = np.column_stack((np.ones((xx.ravel().shape[0], 1)), xx.ravel(), yy.ravel()))
21 X_grid_poly = polynomial(X_grid)
22
23 Z = logistic_function(X_grid_poly.dot(theta_poly))
24 Z = Z.reshape(xx.shape)
25
26 ax.contour(xx, yy, Z, [0.5], colors='blue')
27
28 plt.show()

```

I plotted the decision boundary using the codes above. Using functions, 'meshgrid'

and 'contour', I created the huge matrix X_{grid} consisting of ordered pairs of finely divided x_1 and x_2 values, and I obtained the matrix $X_{grid,poly}$ with 6 features by inserting X_{grid} into the function 'polynomial' I defined. Then, I got output values Z of logistic regression, and I plotted the decision boundary curve in x_1 vs x_2 plane by expressing only the points where the value of Z is 0.5, which means that the dot product of $X_{grid,poly}$ and θ_{poly} is 0.



```
1 p = predict(theta_poly, X_poly)
2 print("Training Accuracy:", sum(p==y)[0], "%")
```

Training Accuracy: 99 %

```
1 test = np.array([50,79])
2 test = (test - mean_scores)/std_scores
3 test = np.append(np.ones(1), test)
4
5 test_poly = polynomial(test.reshape(1, -1))
6
7 probability = logistic_function(test_poly.dot(theta_poly))
8 print("A test sample who x1 = 50 and x2 = 79 will be in the class of C0 in the probability of ",
9       np.round(probability[0], 2))
```

A test sample who $x_1 = 50$ and $x_2 = 79$ will be in the class of C0 in the probability of [0.88]

The training accuracy improved up to 99% from 89%, that is, the model

misclassified only one sample among 100 samples. Although there is an instruction: draw the decision boundary in the space spanned by the bases, I can't express it as 2D figure. Since X_{poly} has 6 features, to draw the decision boundary in the space spanned by the bases, it needs 5 dimensions axes: x_1 , x_2 , x_1^2 , x_1x_2 , and x_2^2 . It is impossible to express graph in 5D, so I drew the decision boundary only for x_1 and x_2 as the figure above.